

Machine Learning II Project Report

The Analysis of the Coffee Quality Dataset

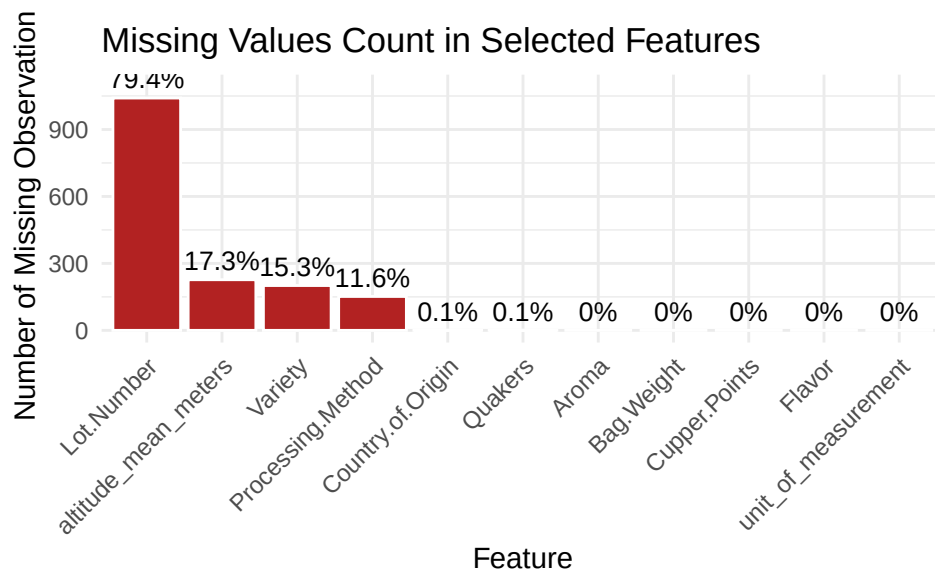
2026-01-02

1 Introduction

This report explores the Coffee Quality dataset, walking through everything from initial descriptive analysis to the math and application of machine learning. Our goal is to use these models to move beyond simple numbers and extract real, meaningful insights into what defines a high-quality cup of coffee.

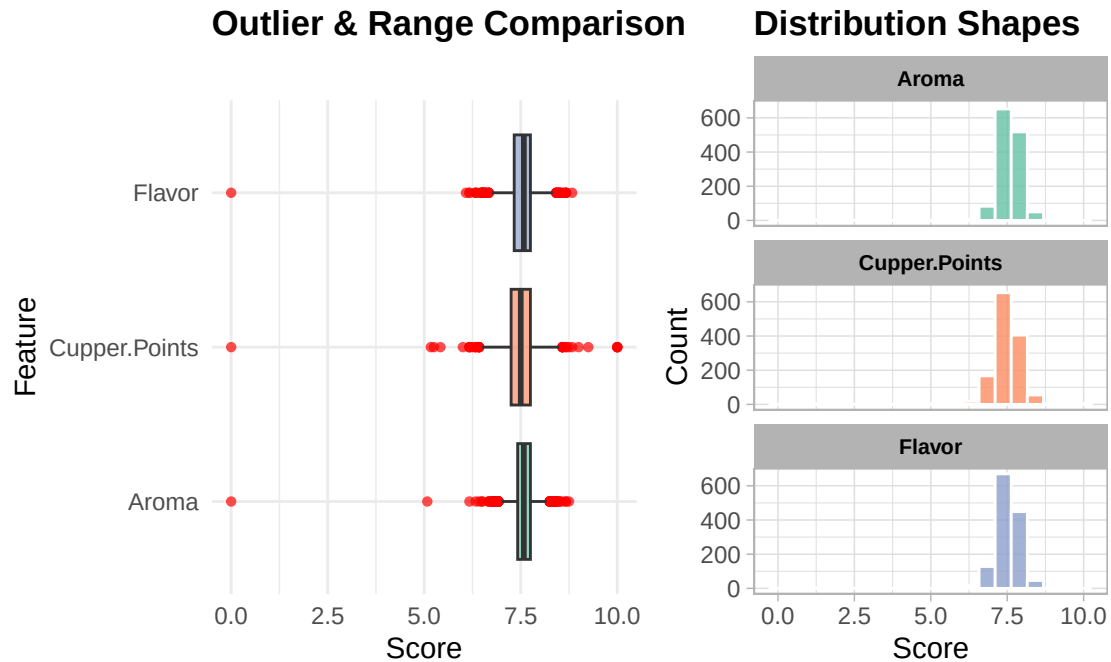
1.1 Descriptive Analysis

Summary of the Descriptive Analysis: The dataset has 1,311 coffee evaluations (observations) with 44 different variables. It's a mix of sensory scores (like how the coffee tastes) and environmental data (like where it was grown). Out of these, **24 variables are categorical and 20 are numerical**. For our project, we decided to use Cupper Points as our target variable. We avoided "Total Cup Points" because that number is just a simple sum of the other scores, which would make the prediction too easy and less meaningful.

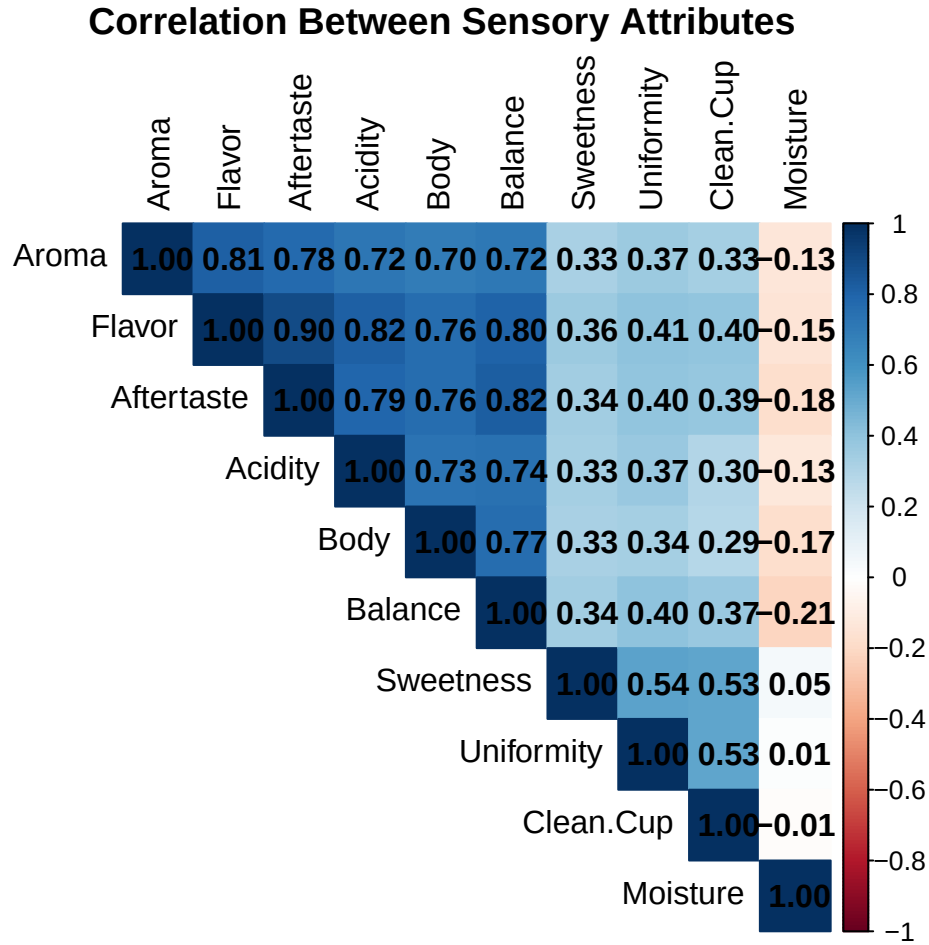


Dealing with Missing Data One of the first things we noticed was that a lot of the categorical (text) data is missing. Because there were so many gaps, it was hard to use many of these variables in our models. Instead, we focused our energy on the variables that really mattered and tried to fill in the blanks for critical info like Altitude.

Sensory Feature Profile: Range and Distribution



The Distribution of the Variables When we looked at the histograms and boxplots for the sensory variables, we saw that they all look very similar. Most of the scores for things like Aroma or Flavor are clustered closely together, usually between 7 and 8. We also noticed several outliers(*coffee samples that scored much higher or lower than the rest*).



Correlation Between Numerical Features Finally, our correlation matrix showed us how these variables lean on each other. Most sensory features are strongly linked. if a coffee smells great (Aroma), it almost always tastes great (Flavor). However, some things, like Moisture, didn't follow the crowd. Moisture has a very low correlation with the other scores and even shows a slight negative trend, meaning more moisture sometimes leads to a slightly lower quality score.

2 Data Preprocessing and Cleaning

Feature selection: In SVM Regression, categorical variables are One-Hot Encoded using dummy variables. This means, features with many unique values will lead to a sparse feature matrix. This is disadvantageous as it increases computational cost and poses the risk of overfitting. Additionally, categories with only a few or even only one observation most likely will not add to model robustness, but are adding noise to the signal. Therefore, we are removing categorical features with many

unique values, only keeping columns with a low number of unique values. E.g. we are not including the “Region” column with 344 unique values in the model training, because we’d on average have less than 4 observations per Region. For features with a high number of missing values, we either remove the column or impute the missing values. For instance for the “Variety” column, we have about 15% missing values and 8% entries “other” at 30 different varieties. Keeping those, would mean adding 30 sparse columns, and at the same time the information gain is most likely not too high with this many missing values. Therefore, we are not including this feature.

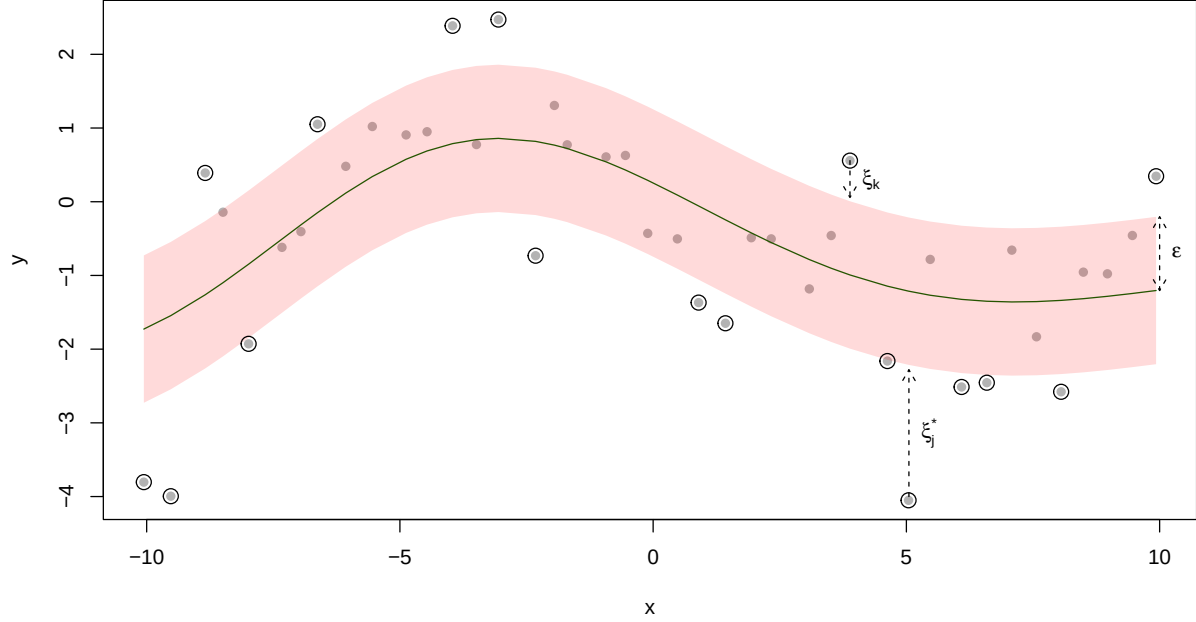
Imputation: For the altitude column, we have about 17% missing values. Here, we are imputing the missing values from the “Region” column. For each observation with missing altitude, we are imputing the median altitude of all coffees from the same region. This covers almost all missing values. For the remaining observations, we are imputing the mean altitude of all coffees from the same country of origin. As the “Region” and “Country” columns are not used in the model training, we are not creating correlated features with this imputation. We are imputing the missing values after the train/test split, using only values from the training set, to avoid data leakage from the test set into the training set.

Further preprocessing: The bag weight column contains both pounds and kilograms, so we are converting all to kilograms. However, for some entries, it is unclear which unit is used. For these, we are averaging the weight in pounds and kilograms. This means for those entries the weight is certainly incorrect, but should be within an order of magnitude of the correct weight. If we can assume that speciality coffee is sold in smaller quantities, we can still gain valuable information from the weight.

Scaling: The SVM method used for model training has a built in scaling functionality. Both predictor and target variables are scaled to have expected value 0 and variance 1. [<https://rdr.io/rforge/e1071/man/svm.html>] This transformation is based on the training data and applied to train and test data sets.

3 A Brief Mathematical Overview of the ML Methods Used

3.1 Support Vector Machine for Regression



Support Vector Regression expands support Vector Machines from discrete classification to a continuous regression model.

For the regression task, instead of fitting a hyperplane, we are trying to fit a (multidimensional) function $f(x)$ to the data. Similar as in the classification task, we draw a margin around the function. However, now the margin ϵ is fixed at the start. (Note that ϵ here, is not the slack variable as in Hasties and Tibshirani, but rather defines the width of the margin. Also, we don't aim to optimize the margin anymore, but instead fix it before the computation starts). We want to find a function $f(x)$ that is as flat as possible, while fitting all points into the required margin. Again, we allow for some slack using slack variables. A wider margin decreases the risk of overfitting, while a smaller margin captures more intricacies of the data.

Mathematically speaking, want to find a function $f(x)$ with $|f(x_i) - y_i| \leq \epsilon$ for all i . To avoid overfitting, we also introduce the constraint of making the function as flat as possible. “Flatness” in this context is a measure of how sensitive the function is to change. For a linear function $f(x) = x^\top \beta + b$ increasing this flatness can be expressed through minimizing the norm of the function gradient: $\|\beta\|$. Minimizing the expression $\frac{1}{2}\|\beta\|^2$ leads to the same optimum, while allowing for more elegant mathematical solutions, and is therefore used for computation.

Again, we want to allow for some slack, so as in the SVM for classification, we are introducing slack variables ξ_i and ξ_i^* . These slack variables allow for some room for margin violations. The optimization problem we want to solve is to minimize:

$$\frac{1}{2}\|\beta\|^2 + K \sum_{i=1}^n (\xi_i + \xi_i^*)$$

subject to:

$$y_i - f(x_i) \leq \varepsilon + \xi_i,$$

$$f(x_i) - y_i \leq \varepsilon + \xi_i^*$$

and

$$\xi_i > 0, \xi_i^* > 0 \quad \forall i$$

where K is fixed and β , ξ and ξ^* are variable. The factor K is introduced as cost for regularization purposes. It is chosen a priori to determine how strong the tradeoff between flatness and overfitting should be. A larger K leads to a stronger impact of the term, therefore penalizing stronger margin violations more heavily and resulting in a less flat function curve (higher bias, lower variance). Conversely, a smaller K leads to a flatter curve (lower bias, higher variance).

As in the SVM for classification, this can be generalized to non-linear functions by transforming the features using kernels.

3.2 XGBoost

XGBoost is an algorithm based on gradient boosted trees and second-order approximation (Chen 2016). It is widely used in machine learning due to its flexibility and strong empirical performance. These properties arise from a combination of algorithmic and systems-level optimizations, including efficient handling of sparse input data, approximate tree construction via weighted quantile sketches, and parallelized tree learning. In addition, careful optimization of memory access, data compression, and out-of-core computation enables the method to scale to very large datasets while maintaining computational efficiency.

This section describes the mathematical foundations of the algorithm, namely ensemble methods, regularization, and gradient boosting machines combined with second order approximation.

3.2.1 Tree Ensembles and Regularization

XGBoost models the prediction as an additive ensemble of K functions:

$$\hat{y}_i = \sum_{k=1}^K f_k(x_i), \quad f_k \in \mathcal{F},$$

where each f_k is a regression tree belonging to the function space $\mathcal{F}$ (Chen 2016; James et al. 2021). Each tree acts as a weak learner that partitions the feature space into disjoint regions and assigns a constant prediction to each leaf (James et al. 2021).

The model is trained by minimizing a regularized objective function of the form

$$\mathcal{L} = \sum_i \ell(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k),$$

where $\ell(\cdot)$ is a differentiable loss function and $\Omega(f)$ is a regularization term that penalizes model complexity (Chen 2016). This term is defined as

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2,$$

with T denoting the number of leaves in the tree and w_j the weight of leaf j . The regularization term encourages simpler trees and smooth leaf weights, helping to prevent overfitting.

The ensemble structure allows the model to reduce bias by combining multiple weak learners, while regularization controls variance by limiting tree complexity and the magnitude of leaf weights (James et al. 2021). This balance between bias and variance enables XGBoost to achieve strong predictive performance while maintaining good generalization.

3.2.2 Gradient Boosting Machines

The tree ensemble is constructed in a stage-wise manner, where weak learners are added iteratively to improve the model (Friedman 2001). At iteration t , the prediction is given by

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + f_t(x_i),$$

with $f_t \in \mathcal{F}$ denoting the newly added regression tree.

Gradient tree boosting can be interpreted as gradient descent in function space, where each new learner is chosen to minimize the loss function in the direction of the negative gradient (Friedman 2001). Specifically, the new tree at iteration t is trained to fit the negative first order gradient

$$-g_i^{(t)} = - \left. \frac{\partial \ell(y_i, \hat{y})}{\partial \hat{y}} \right|_{\hat{y}=\hat{y}_i^{(t-1)}}.$$

XGBoost extends this framework by explicitly approximating the objective function using a second-order Taylor expansion of the loss around the current predictions:

$$\ell(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) \approx \ell(y_i, \hat{y}_i^{(t-1)}) + g_i f_t(x_i) + \frac{1}{2} h_i f_t(x_i)^2,$$

where

$$h_i = \left. \frac{\partial^2 \ell(y_i, \hat{y})}{\partial \hat{y}^2} \right|_{\hat{y}=\hat{y}_i^{(t-1)}}$$

is the second order derivative of the loss (Chen 2016). By incorporating both first- and second-order information, XGBoost enables efficient optimization of tree structures and leaf weights within each boosting iteration.

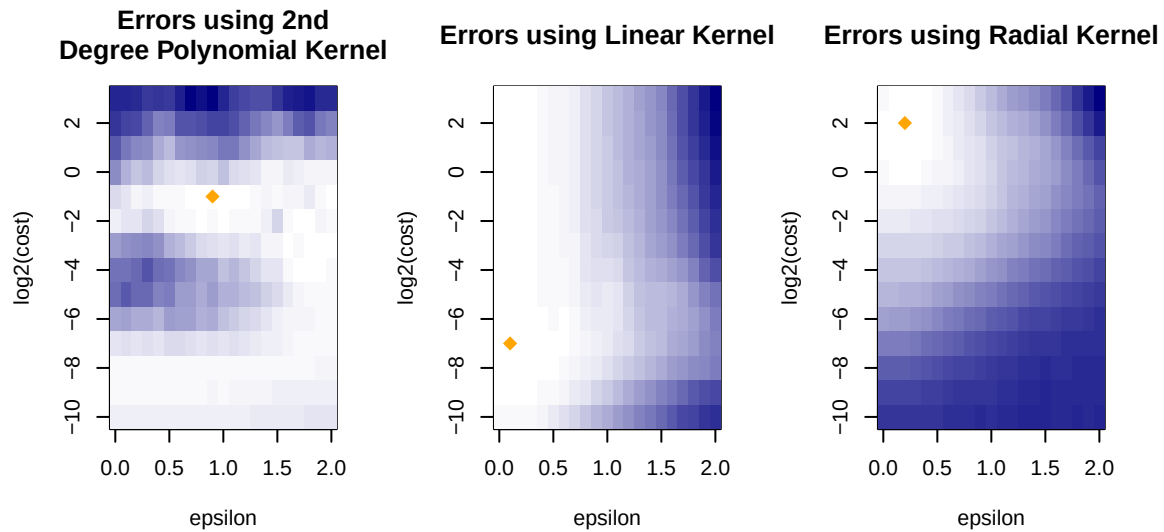
4 Applications

4.1 Support Vector Regression

4.1.1 Support Vector Regression - Hyperparameter Optimization

For the prediction of copper points we train an SVM for regression on the data set, holding out the test data set for final evaluation. To choose appropriate hyperparameters, we conduct a hyperparameter optimization on the training data set using 10-fold cross-validation. We use a grid

search for different values for cost and epsilon for a radial, linear, and 2-degree polynomial kernel. We compare the mean absolute error between runs to choose the best-performing hyperparameter combination. Finally, we train a model with these hyperparameters on the entire training set and evaluate on the test data set.



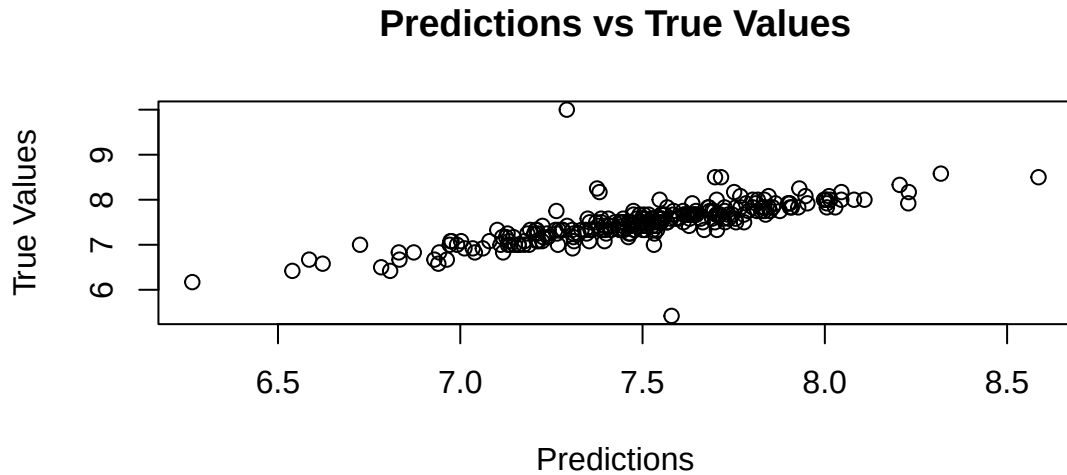
These figures show the mean absolute error for the different hyperparameters by kernel. The best performing hyperparameter combination for each kernel are marked with orange points.

```
## Best SVM kernel and parameters selected:
```

```
##   kernel epsilon      cost
## 2 linear      0.1 0.0078125
```

```
## [1] "MAE: 0.143099275160558"
```

The best performing kernel is the linear kernel, which means the relationship between the features and the target variable is best approximated by a multidimensional hyperplane.



Here, we can see the predictions vs. the true values for the test data set. We can see that aside from two outliers, the predictions are close to the true values.

4.2 XGBoost

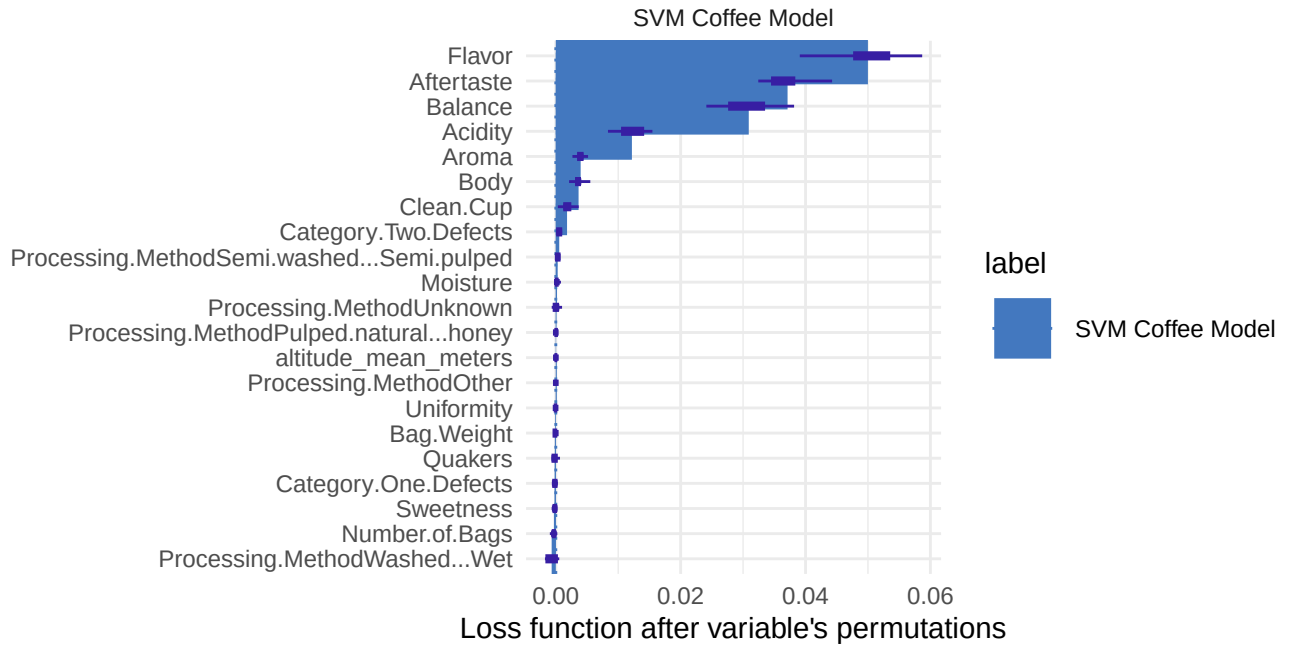
5 XAI and Comparison of the Models SVR and XGBoost

5.1 Permutation Feature Importance (PFI)

FOR SVM PFI; The chart shows us exactly which “ingredients” the model thinks are most important for a high-quality cup of coffee. Flavor and Aftertaste are the clear winners here; when the model loses track of these, its accuracy drops the most. Balance and Acidity also help quite a bit, but they aren’t as critical as the overall taste. Interestingly, all the “technical” details—like how much the Bag Weighs, the Altitude where it was grown, or the Moisture level don’t effect significantly. Basically, the model has learned that if the coffee tastes great and has a clean finish, the technical background don’t change the final score very much.

Feature Importance (PFI) for Coffee Quality

How much the error increases when a feature is shuffled

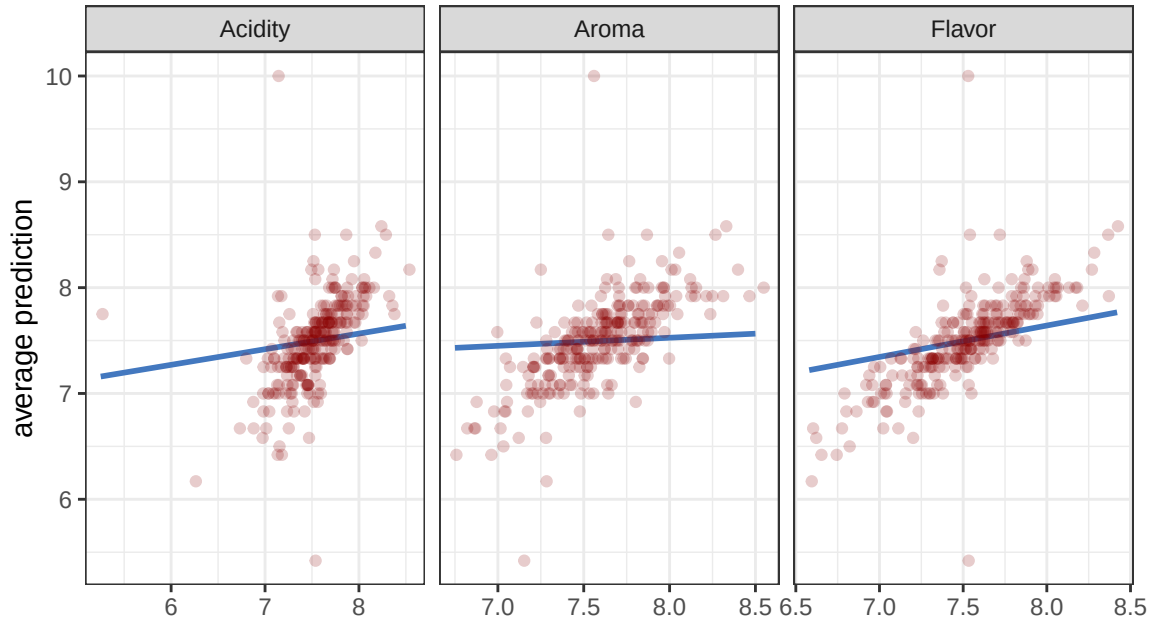


5.2 Partial Dependence Plot (PDP)

FOR SVM PDP; The plots show how the model's quality predictions change as we look at different scores for *Acidity*, *Aroma*, and *Flavor*. The upward-sloping blue lines tell a very logical story: as these sensory scores go up, the model's predicted quality score also rises. By adding the red dots (our actual coffee samples), we can see that the model is doing a good job of finding the "middle ground" where most of the data lives. Also, we can see that the model doesn't get distracted by the outliers, the model stays focused on the general trend, which shows it has built a very stable and reliable "logic" for what makes a good cup of coffee.

Partial Dependence Plots for Coffee Quality

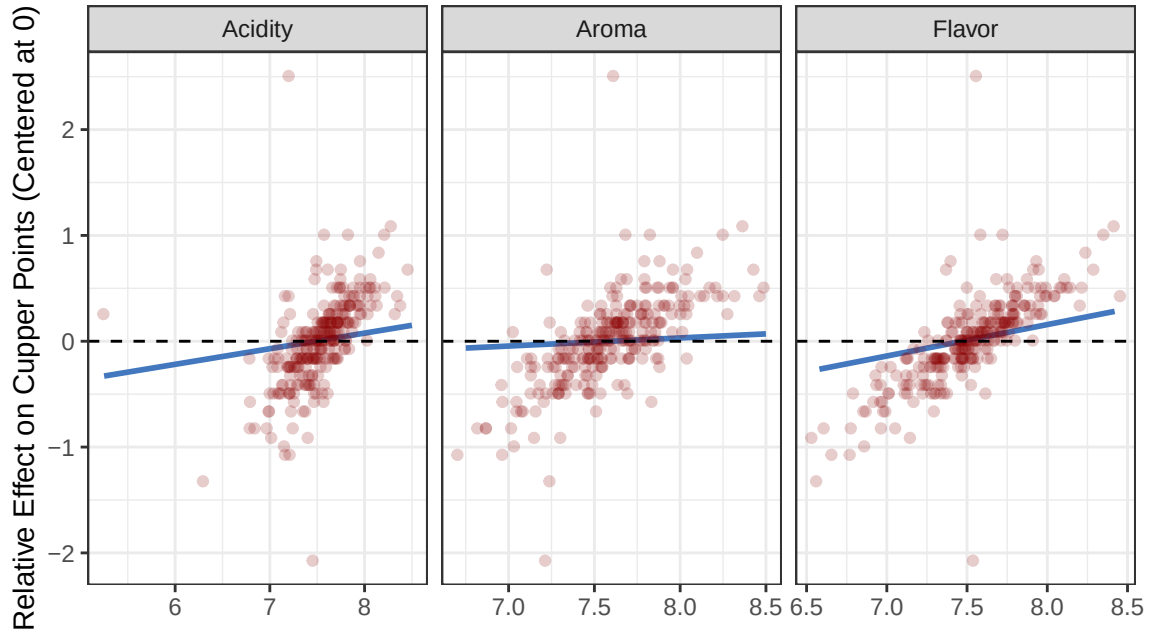
Red dots = Actual observations | Blue line = Model average



5.3 Accumulated Local Effects (ALE)

ALE is a better option to see the feature importance in a model who has correlated features. Because The ALE plot is just a way to see how one specific feature (like Flavor) changes the final score, without getting confused by other features. **FOR SVM ALE;** The dashed line at zero represents an average cup of coffee in our data. When the blue line is above this zero mark, it means that specific feature is adding “bonus points” to the quality, while anything below zero means it is pulling the score down. Because the blue line slopes upward to the right, we can see the model correctly learned that higher sensory scores lead to higher quality predictions. The red dots represent the actual coffee samples, and since they cluster tightly around the blue line, it proves the model’s internal logic is a very strong match for the real-world results as we have seen in PDP plots as well.

Aligned Centered ALE Plots
Created for the SVM Coffee Model model

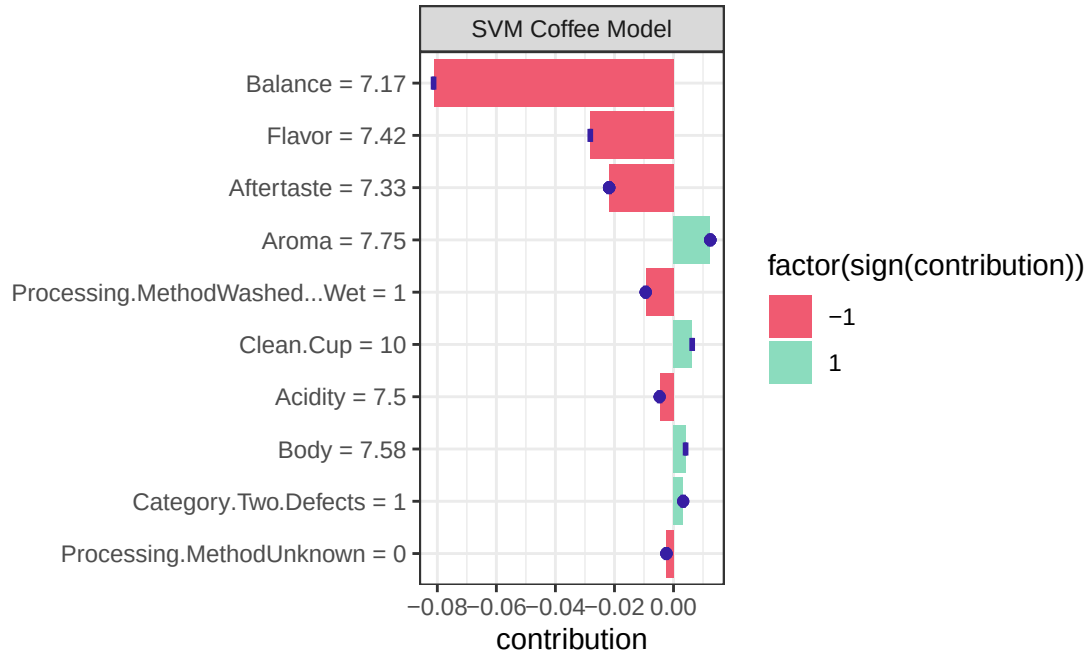


5.4 Shapley Additive Explanations (SHAP)

The SHAP is a Local Explanation, which acts like a “scorecard” for one specific cup of coffee to show why it received its particular prediction.

FOR SVM SHAP; In this specific example (*the number of observation is 100*), the model’s prediction was dragged down primarily because the Balance, Flavor, and Aftertaste scores were lower than average, as shown by the red bars extending to the left. On the other hand, the green bars show that this coffee’s Aroma and Clean Cup scores actually helped boost its final rating. Essentially, this plot shows exactly which characteristics made this specific cup of coffee better or worse than a typical sample.

Local Explanation for a Single Coffee Sample Contribution of each feature to the specific Cupper Po



6 Literature

- Chen, Tianqi. 2016. "XGBoost: A Scalable Tree Boosting System." *Cornell University*.
- Friedman, Jerome H. 2001. "Greedy Function Approximation: A Gradient Boosting Machine." *Annals of Statistics*, 1189–1232.
- James, Gareth, Daniela Witten, Trevor Hastie, and Robert Tibshirani. 2021. *An Introduction to Statistical Learning: With Applications in R*. 2nd ed. Springer. <https://doi.org/10.1007/978-1-0716-1418-1>.